Lab 6 Report: Code and Branch Coverage Analysis

PES1UG24AM321 · 29 October 2026

New tests raised coverage of `atm_withdrawal.py` from 41% to 100% line coverage and 100% branch coverage (22 of 22 branches).

Item	Detail
SRN	PES1UG24AM321
Assigned repository	SETAPESU26/L6_L7_18_atm_withdrawal
Submission repository	Vocod3r/Software_engineering_Lab_PES1UG24AM321 (folder `SE_LAB6_PES1UG24AM321`)
Application under test	`atm_withdrawal.py`
Test files	`CodeCoverageTest_atm_withdrawal.py`, `BranchCoverageTest_atm_withdrawal.py`
Environment	Windows, Python 3.11.9, pytest 9.1.1, coverage 7.17.0

Objective

The lab measures how much of a program its tests exercise, using two metrics:

- **Line (code) coverage**: the share of statements executed at least once.
- **Branch coverage**: the share of decision outcomes (each `if` going True and False) executed.

The target was at least 90% for each. Tests were written with pytest and measured with Python's `coverage` library.

Application under test

`atm_withdrawal.py` has 34 statements and 22 branches across three functions.

Function	What it does	Decisions to cover
`withdraw(balance, amount, account_type, own_bank)`	Validates the account and amount, applies the daily limit (savings 25,000, current 50,000), adds an ATM fee (20 at another bank, +10 above 10,000), checks the balance	Unknown account; amount ≤ 0 or not a multiple of 100; over limit; own bank or not; amount > 10,000; amount + fee > balance
`pin_check(entered, actual, attempts)`	Returns `ok`, `retry` or `card blocked`	attempts ≥ 3; PIN correct; wrong PIN on attempt 2
`note_breakup(amount)`	Splits an amount into 500, 200 and 100 notes	Not a multiple of 100; remainder ≥ 200

Baseline coverage

The two starter tests covered 41% of `atm_withdrawal.py`: 20 of 34 statements never ran. Only a successful own-bank withdrawal and a correct PIN were tested, so every error path, the fee logic and all of `note_breakup` were missed.

```
coverage run -m pytest test_atm_withdrawal.py
coverage report -m
```

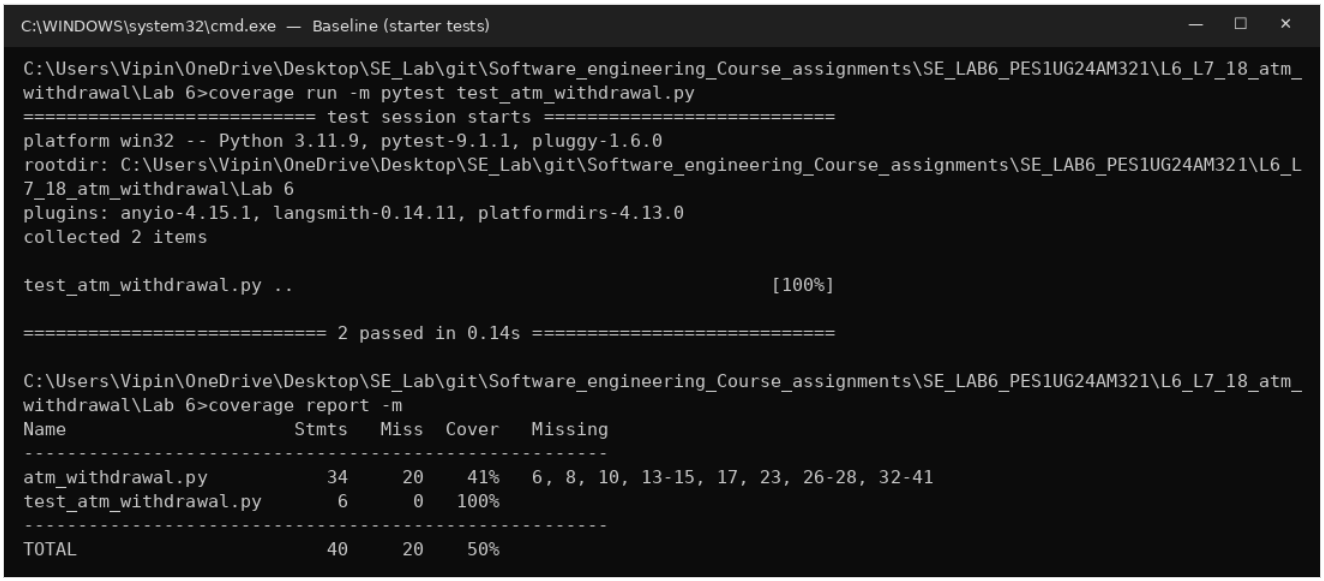


Figure 1. Baseline coverage with the starter tests: 41%

Code (line) coverage

`CodeCoverageTest_atm_withdrawal.py` reached 100% line coverage with 13 passing tests: all 34 statements ran.

Each test targets lines the baseline missed:

Function	Tests added	Lines reached
`withdraw`	Unknown account type; amount 0; amount above the savings limit; 20,000 at another bank (fee 30); balance too low	Both `ValueError` raises, `limit exceeded`, the fee block, `insufficient funds`
`pin_check`	Attempts = 3; wrong PIN on attempt 2; wrong PIN on attempt 0	Both `card blocked` returns, `retry`
`note_breakup`	150 (invalid); 1800 → (3, 1, 1); 600 → (1, 0, 1)	The `ValueError`, the 200-note branch, the `else` branch

```
coverage run -m pytest CodeCoverageTest_atm_withdrawal.py -v
coverage report -m --include=atm_withdrawal.py
```

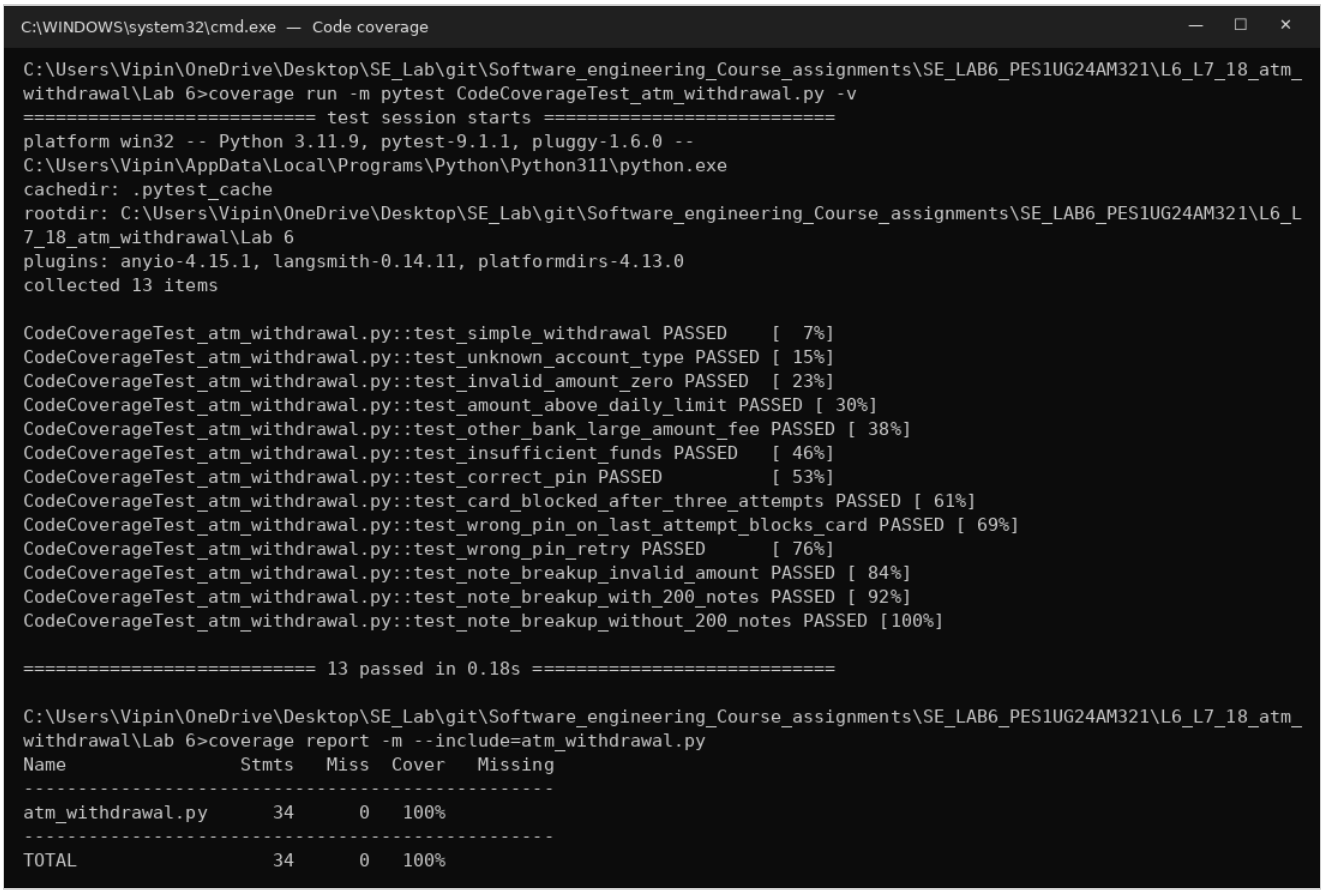


Figure 2. Code coverage: 13 tests passed, 100% line coverage

Branch coverage

`BranchCoverageTest_atm_withdrawal.py` reached 100% branch coverage with 47 passing tests: 22 of 22 branches taken, 0 partial.

Every decision is driven both ways, with boundary values at each threshold:

Decision	True case	False case	Boundary checked
`account_type not in DAILY_LIMIT`	`fixed`, `SAVINGS`, empty, `None`	`savings`, `current`	Case sensitivity
`amount <= 0 or amount % 100 != 0`	0, -100, 99, 150, 1050	100	Smallest valid amount
`amount > DAILY_LIMIT`	25,100 savings; 50,100 current	25,000 savings; 50,000 current	Exactly at each limit
`not own_bank`	Other-bank withdrawals	Own-bank withdrawals	—
`amount > 10000` (fee)	10,100 → fee 30	1,000 and 10,000 → fee 20	Exactly 10,000
`amount + fee > balance`	1,000 from 500; 1,000 + fee from 1,000	1,000 + 20 from 1,020	Balance exactly covers amount + fee
`attempts >= 3`	3, 4, 10 (even with correct PIN)	0, 1, 2	Exactly 3
`entered == actual`	Correct PIN, attempts 0–2	Wrong PIN	—

Decision	True case	False case	Boundary checked
`attempts == 2`	Wrong PIN on attempt 2	Wrong PIN on attempts 0, 1	—
`amount % 100 != 0` (notes)	50, 150, 1999	Valid amounts	—
`remaining >= 200`	200, 300, 400, 1800, 2900	0, 100, 500, 600, 2500	Remainder exactly 200

One extra test checks every amount from 0 to 10,000 in steps of 100: the notes always add back up to the amount.

```
coverage run --branch -m pytest BranchCoverageTest_atm_withdrawal.py -v
coverage report -m --include=atm_withdrawal.py
```

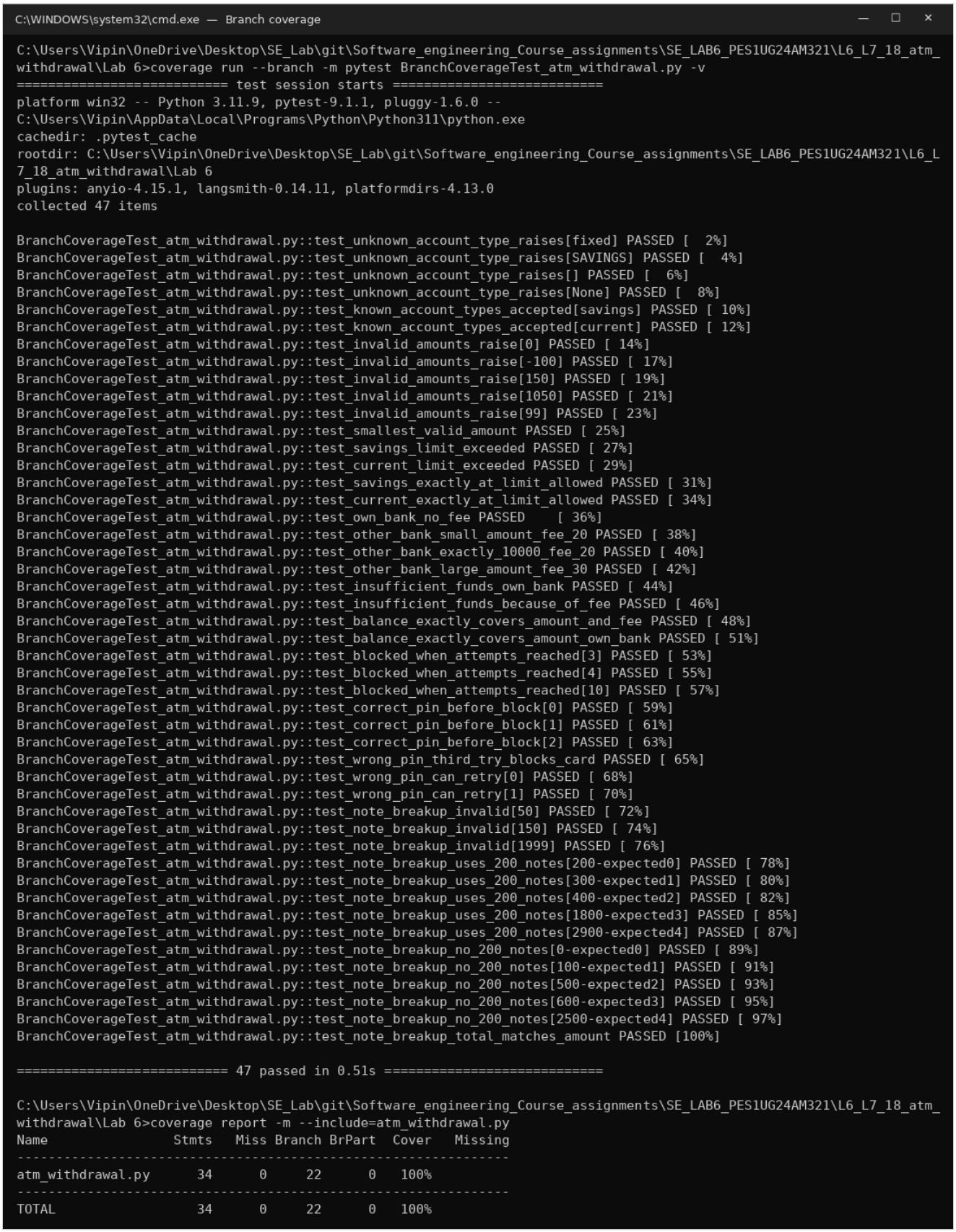


Figure 3. Branch coverage: 47 tests passed, 22 of 22 branches, 100%

Line vs branch coverage

100% line coverage did not mean 100% branch coverage: the line-coverage tests, measured with `--branch`, scored 98% with one partial branch.

Test file	Tests	Line coverage	Branch coverage
`test_atm_withdrawal.py` (starter)	2	41%	38%
`CodeCoverageTest_atm_withdrawal.py`	13	100%	98% (1 partial)
`BranchCoverageTest_atm_withdrawal.py`	47	100%	100%

The missed branch is the fee check `if amount > 10000` going False. The line tests only withdrew 20,000 at another bank, so every line of the fee block ran, but a small other-bank withdrawal (fee 20 only) was never tested. A bug there would pass line coverage unnoticed.

Branch coverage also pushed the tests onto boundary values (exactly 10,000, exactly the daily limit, a balance that exactly covers amount + fee). These are where off-by-one mistakes such as `>` vs `>=` show up.

Commits

The work is in `SE_LAB6_PES1UG24AM321/L6_L7_18_atm_withdrawal/Lab 6` of the submission repository.

Commit	Message	Contents
`7251291`	Lab 6: code coverage tests (100% line coverage) + screenshot	Application, starter tests, `CodeCoverageTest_atm_withdrawal.py`, `code_coverage_report.png`
`acb6b72`	Lab 6: branch coverage tests (100% branch coverage) + screenshot	Removed generated `.coverage` and `__pycache__` files

☐ Commit and push `BranchCoverageTest_atm_withdrawal.py` and `branch_coverage_report.png`, then add that commit here.

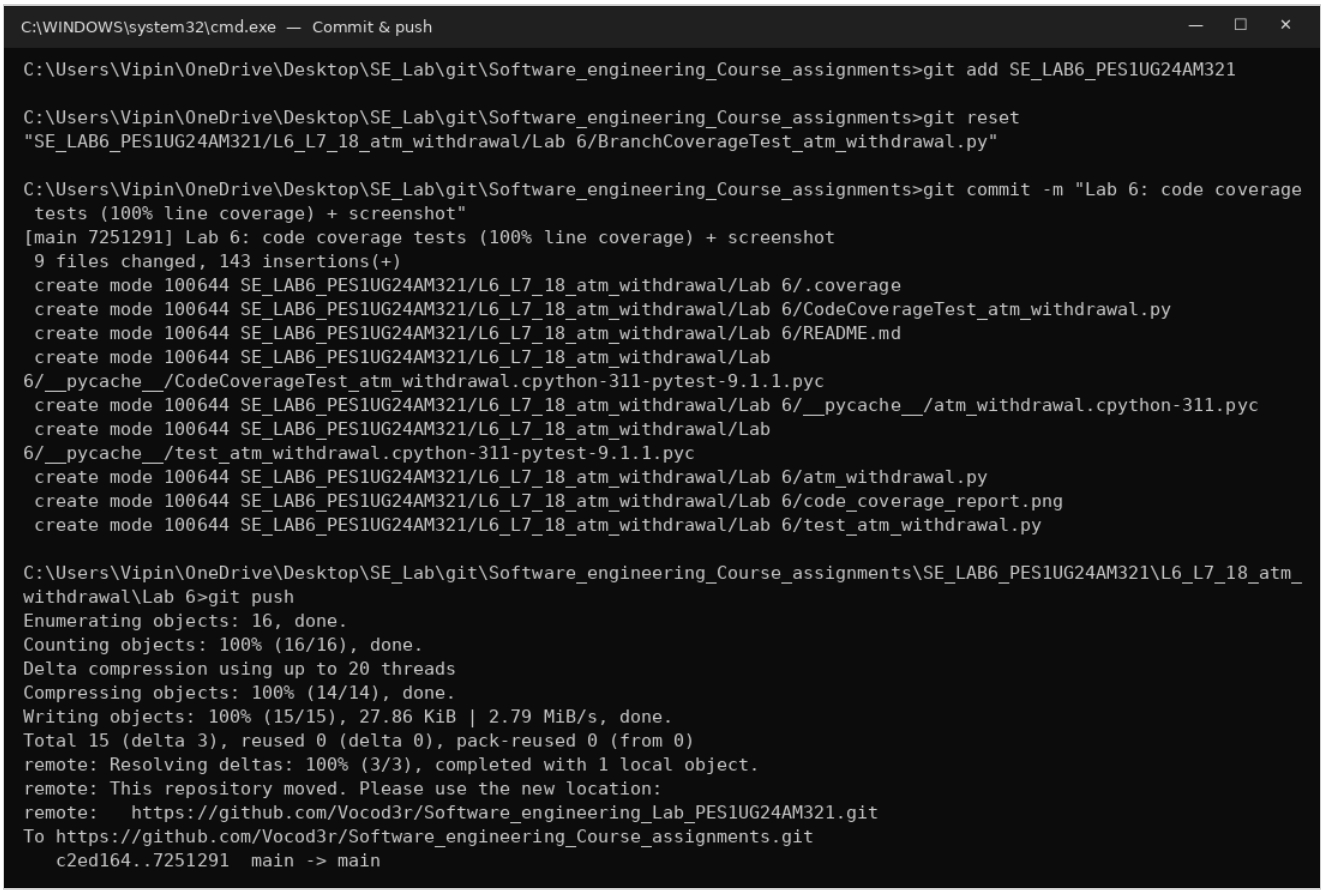


Figure 4. First commit and push

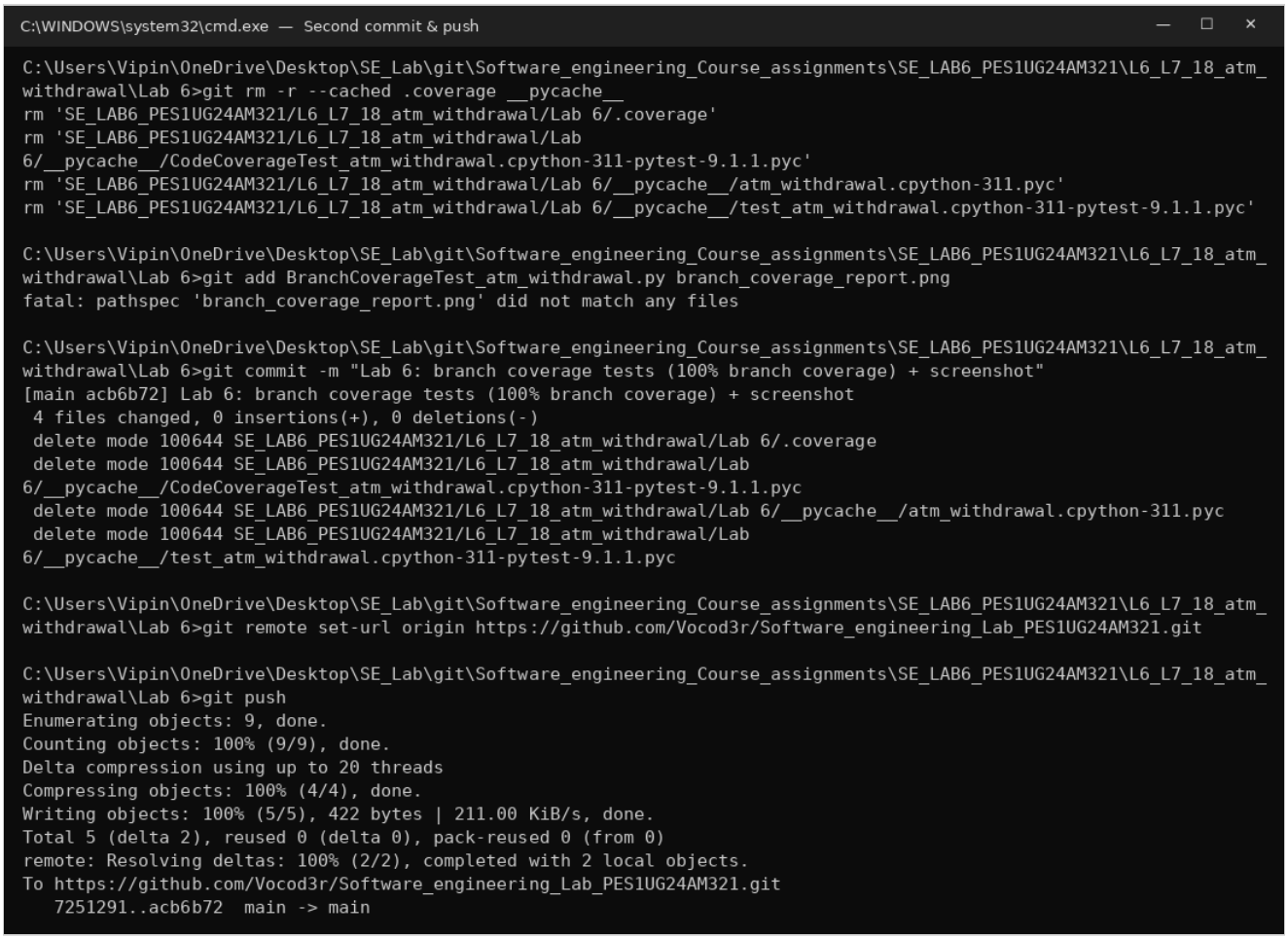


Figure 5. Second commit and push

Conclusion

Both targets were exceeded: 100% line and 100% branch coverage, up from 41% and 38%. Line coverage alone left a fee path untested; branch coverage closed it and added boundary tests at every threshold. All 60 new tests pass.